

RISCV - 8×8 GEMM

8×8 GEMM 연산을 위한 RISC-V 기반 프로세서 구조 개선 및 PPA 분석

1 조

차상훈

0. 목차

1. 전체 프로세서 아키텍처 설명
2. Baseline 대비 주요 변경 사항
3. Assembly 코드 최적화 전략
4. Datapath 및 control path 개선 내용
5. Performance, energy, area 결과 분석
6. 설계 trade-off 분석
7. 제약 조건 준수 여부
8. 결론

1. 전체 프로세서 아키텍처 설명

본 프로젝트는 RISC-V 기반 프로세서에서 8x8 GEMM(General Matrix Multiplication) 연산을 수행하도록 datapath 와 control path 를 개선하는 것을 목표로 한다. GEMM 은 A 와 B 두 행렬을 곱하여 C 행렬을 생성하는 연산이며, 본 설계에서는 A 와 B 를 8-bit unsigned data 로 처리하고 C 는 32-bit word 단위로 저장한다.

전체 구조는 별도의 외부 GEMM accelerator 를 추가하는 방식이 아니라, instruction memory 에서 명령어를 fetch 하고 CPU 내부 datapath 를 통해 명령어를 실행하는 일반적인 RISC-V instruction execution flow 를 유지한다. TOP_GEMM 은 CPU core 인 RISCVMULTI 와 instruction memory, data memory 를 연결하는 wrapper 역할을 하며, 실제 pipeline 동작은 DATAPATH 모듈 내부에서 수행된다.

구성 요소	역할
TOP_GEMM	CPU core, IMEM, DMEM 을 연결하는 최상위 검증용 wrapper
RISCVMULTI	외부 메모리 인터페이스를 갖는 5-stage pipeline CPU core
DATAPATH	IF/ID/EX/MEM/WB pipeline register 와 ALU, branch, forwarding, WB 경로 포함
CONTROLLER	opcode, funct3, funct7 정보를 이용해 control signal 생성
MAINDEC / ALUDEC	opcode 별 주요 제어 신호 및 ALU 연산 제어 생성
REGFILE	32 개의 32-bit register file, x0 hardwired zero, write-first read 지원
BRANCH_PRED	branch taken 여부를 예측하고 misprediction 시 flush 복구와 연동
IMEM / DMEM	instruction 저장 및 A/B/C/STATUS data memory 역할

프로세서 구조는 5-stage pipeline 으로 구성된다. 각 stage 는 Instruction Fetch(IF), Instruction Decode/Register Fetch(ID), Execute(EX), Memory Access(MEM), Write Back(WB)로 구분된다. 각 stage 사이에는 IF/ID, ID/EX, EX/MEM, MEM/WB pipeline register 가 존재하며, instruction, operand, immediate, control signal 을 다음 stage 로 전달한다.

Stage	주요 동작
IF	PC 를 이용해 instruction memory 에서 명령어를 읽고, PC+4 또는 branch prediction target 을 계산한다.
ID	명령어를 decode 하고 rs1/rs2/rd 를 추출하며 register file read 와 immediate extension 을 수행한다.
EX	ALU 연산, branch target 계산, branch taken 판단, forwarding mux 선택을 수행한다.
MEM	data memory read/write 를 수행하고 A/B 영역 read data 에 대해 8-bit masking 을 적용한다.
WB	ALU result 또는 memory read data 를 register file 에 writeback 한다.

Memory map 은 과제의 testbench 와 호환되도록 유지하였다. A matrix 는 0x000, B matrix 는 0x100, C matrix 는 0x200, STATUS 는 0x300 주소에 배치된다. 연산 완료 후에는 STATUS 주소에 1 을 저장하여 testbench 가 완료 시점을 판단할 수 있도록 한다.

항목	시작 주소	설명
----	-------	----

Matrix A	0x000	8×8 입력 행렬 A
Matrix B	0x100	8×8 입력 행렬 B
Matrix C	0x200	8×8 결과 행렬 C
STATUS	0x300	연산 완료 시 1 저장

2. Baseline 대비 주요 변경 사항

Baseline 구조는 GEMM 을 단순한 software loop 로 수행하거나, pipeline 및 branch/hazard 최적화가 충분하지 않은 구조로 볼 수 있다. 본 설계는 GEMM workload 의 실행 패턴에 맞춰 프로세서 구조와 assembly code 를 함께 변경하였다. 핵심 변경 사항은 5-stage pipeline 적용, branch prediction 적용, ALU 연산 case 축소, prefix adder 기반 ALU 최적화, 4×4 tile assembly 최적화이다.

구분	Baseline	개선 구조
명령어 처리 구조	단순 순차 실행 또는 기본 pipeline	IF/ID/EX/MEM/WB 5-stage pipeline
GEMM 곱셈	반복 덧셈 또는 단순 shift-add	bit 기반 shift-add + 4×4 tile 누산 구조
분기 처리	branch 결과 확정 전 penalty 발생	branch predictor 와 flush control 적용
Data hazard 처리	stall 또는 비최적 명령어 배치	instruction scheduling 으로 forwarding unit 완전 제거
ALU 구조	범용 연산 case 포함 가능	GEMM 에 필요한 ADD/SUB/AND 중심으로 축소
PPA 관점	cycle 수와 실행시간 증가	cycle 수 감소 및 energy 감소 목표

ALU 는 전체 RV32I 명령어를 모두 지원하는 방향이 아니라, 제공된 GEMM assembly 가 실제 사용하는 명령어 subset 에 맞추어 단순화하였다. ALUDEC 는 ADD, SUB, AND 만 생성하며, OR, SLT, shift 계열은 제거하였다. 이는 control path 와 ALU mux case 를 줄여 area 와 timing 을 개선하기 위한 선택이다.

3. Assembly 코드 최적화 전략

GEMM 은 8×8 결과 C 의 각 원소를 계산하기 위해 8 개의 곱셈과 누산을 수행한다. 따라서 전체 연산에는 512 회의 곱셈이 필요하며, 이를 단순 반복덧셈으로 구현하면 cycle 수가 크게 증가한다. 본 설계에서는 MUL 명령어를 사용하지 않고 RV32I subset 명령어만으로 곱셈을 수행하기 위해 bit 기반 shift-add 알고리즘을 사용하였다.

핵심 아이디어는 B 값의 각 bit 를 검사하고, 해당 bit 가 1 이면 현재 bit 위치에 해당하는 A 의 shift 값을 결과 누산기에 더하는 것이다. 예를 들어 B 의 bit mask 가 1, 2, 4, 8, ...로 증가할 때, A 역시 add x8, x8, x8 과 같은 명령어로 두 배씩 증가한다. 즉, 실제 shift instruction 을 사용하지 않고 add 연산으로 shift 효과를 만든다.

```
# B 의 현재 bit 검사 후, bit 가 1 이면 해당 A 값을 누산기에 더함
and x7, x12, x2
```

```

beq x7, x0, t0_c0_skip
add x16, x16, x8
add x20, x20, x9
add x24, x24, x10
add x28, x28, x11

```

다음 bit 계산을 위해 A 값과 mask 를 2 배로 증가

```

add x8, x8, x8
add x9, x9, x9
add x10, x10, x10
add x11, x11, x11
add x2, x2, x2

```

Assembly 는 4×4 tile 단위로 최적화되었다. x16 부터 x31 까지 총 16 개의 register 를 결과 누산기로 사용하여 하나의 4×4 결과 tile 을 동시에 누적한다. 먼저 16 개의 누산기를 0 으로 초기화하고, A 의 4 개 row 값과 B 의 4 개 column 값을 load 한 뒤, B 의 bit 검사 결과에 따라 네 column 에 대한 누산을 병렬적으로 수행한다.

최적화 항목	내용	효과
16 개 누산기 사용	x16~x31 을 4×4 tile 결과 누산기로 사용	C 원소 16 개를 register 에 유지하여 memory 접근 감소
bit 기반 shift-add	B 의 각 bit 를 검사하고 shift 된 A 값을 누산	MUL 없이 곱셈 구현, 반복덧셈 대비 cycle 감소
4×4 tile 처리	4 개 row × 4 개 column 을 하나의 block 으로 처리	loop overhead 와 pointer update 감소
pointer arithmetic	A/B/C pointer 를 block 단위로 갱신	주소 계산 명령어 수 감소
register reuse	x6 등 일부 register 를 필요한 시점에 재사용	register pressure 와 불필요한 이동 감소

이 방식은 instruction 수가 일부 증가할 수 있지만, 반복덧셈 기반 곱셈에 비해 cycle 수를 크게 줄일 수 있다. 특히 4×4 tile 구조는 결과를 memory 에 매번 저장하지 않고 register 에 누산한 뒤 tile 계산이 끝난 후 한 번에 저장하므로 memory 병목을 줄이는 장점이 있다.

4. Datapath 및 Control Path 개선 내용

Datapath 개선의 첫 번째 핵심은 5-stage pipeline 구조이다. DATAPATH 모듈은 IF/ID, ID/EX, EX/MEM, MEM/WB pipeline register 를 포함하며, 각 stage 의 instruction 정보와 control signal 을 다음 stage 로 전달한다. 이를 통해 하나의 instruction 이 모든 과정을 끝낼 때까지 기다리지 않고, 여러 instruction 이 동시에 처리될 수 있다.

두 번째는 forwarding unit 제거 및 instruction scheduling 적용이다. 초기 설계에서는 EX/MEM→EX forwarding path 를 통해 RAW hazard 를 처리하였으나, forwarding mux 가 ALU 입력 앞에 위치하여 critical path 를 연장하는 문제가 발생하였다. 이를 해결하기 위해 assembly instruction 의 배치를

재조정하여 and 명령어 3 개를 beq 보다 먼저 실행함으로써 gap 을 3 이상으로 확보하였다. 이를 통해 forwarding 없이도 register file 에서 최신 값을 직접 읽을 수 있게 되어 forwarding unit 을 완전히 제거하였다. forwarding unit 제거로 ALU 입력 경로가 단순화되고 critical path 가 단축되어 clock period 를 2.55ns 로 줄일 수 있었다.

```

wire [31:0] ex_srca = (fwd_a == 2'b10) ? exmem_aluresult :
                    (fwd_a == 2'b01) ? wb_result      :
                    idex_rd1;

wire [31:0] ex_fwd_b = (fwd_b == 2'b10) ? exmem_aluresult :
                    (fwd_b == 2'b01) ? wb_result      :
                    idex_rd2;

```

세 번째는 branch prediction 과 flush control 이다. IF stage 에서는 branch predictor 가 branch taken 여부를 미리 예측하고, 예측이 taken 인 경우 predicted target 을 next PC 로 선택한다. 이후 EX stage 에서 실제 branch 결과가 결정되며, 예측이 틀렸거나 JAL instruction 이 발생하면 flush 를 통해 잘못 fetch 된 instruction 을 제거하고 correct PC 로 복구한다.

```

wire [31:0] if_pc_next = ex_flush  ? ex_pc_correct :
                        pred_taken ? pred_target  :
                        if_pc_plus4;

assign ex_flush      = ex_mispred | ex_jump_taken;
assign ex_pc_correct = ex_take ? ex_pc_target : ex_pc_plus4;

```

네 번째는 A/B 8-bit masking 이다. A 와 B 는 8-bit unsigned input 이지만, CPU 는 32-bit lw 명령어로 memory 를 읽는다. 따라서 MEM stage 에서 주소가 A/B 영역인 경우 READDATA 의 하위 8-bit 만 사용하고 상위 24-bit 는 0 으로 확장하였다. 이를 통해 lbu 명령어 없이도 8-bit 입력 조건을 만족할 수 있다.

```

wire      mem_in_ab      = (exmem_aluresult_r < 32'h200);
wire [31:0] mem_rd_masked = mem_in_ab ? {24'b0, READDATA[7:0]} : READDATA;

```

Control path 는 MAINDEC 와 ALUDEC 로 분리하였다. MAINDEC 는 opcode 를 보고 RegWrite, MemWrite, ALUSrc, ResultSrc, Branch, Jump, ImmSrc, ALUOp 를 생성한다. ALUDEC 는 ALUOp 와 funct3/funct7 정보를 기반으로 ALUControl 을 생성한다. 본 설계에서는 LW, SW, R-type ADD/SUB, BEQ, I-type ADDI/ANDI, JAL 중심으로 control case 를 구성하였다.

명령어	주요 Control 동작
LW	RegWrite=1, ALUSrc=1, ResultSrc=Memory, ALU=ADD
SW	MemWrite=1, ALUSrc=1, ALU=ADD

ADD/ADDI	RegWrite=1, ResultSrc=ALU, ALU=ADD
SUB/BEQ	BEQ 는 ALU=SUB 로 zero 비교 후 branch 판단
AND/ANDI	bit mask 연산용 AND 수행
JAL	PC target 으로 jump, PC+4 를 writeback 경로로 전달

5. Performance, Energy, Area 결과 분석

PPA 분석은 네 가지 설계 단계에 대해 수행하였다. 첫 번째는 shift-add 만 적용한 구조, 두 번째는 4x4 tile 을 적용한 구조, 세 번째는 instruction 재배치 및 forwarding unit 제거를 적용한 구조, 네 번째는 branch predictor 를 추가 최적화한 최종 구조이다. 표의 수치는 발표 자료의 PPA 결과를 기준으로 정리하였다.

구조	Clock Period	Power	Area	Average Cycle	Execution Time	Energy
Shift-add 적용	3.0 ns	3.36 mW	31186.1	12856	38 μ s	129 nJ
4x4 tile 적용	3.0 ns	3.63 mW	35110	3704	11.1 μ s	40.5 nJ
Instruction 재배치 / forwarding unit 제거	2.55 ns	4.16 mW	34272.76	3793	9.67 μ s	40.3 nJ
Branch Predictor 최적화 (최종)	2.40 ns	4.13 mW	32421.42	3793	9.10 μ s	37.6 nJ

Shift-add 만 적용한 구조는 반복뎁셈 대비 개선되었지만, register reuse 와 tile 최적화가 부족하여 average cycle 이 12856 으로 크게 나타났다. 따라서 execution time 은 38 μ s, energy 는 129nJ 로 가장 크다.

4x4 tile 적용 구조는 cycle 수를 3704 로 크게 줄였다. 이는 16 개의 누산기 register 를 사용하여 4x4 결과 tile 을 register 내부에서 누적하고, tile 계산 후 memory 에 저장하기 때문이다. 그 결과 execution time 은 11.1 μ s, energy 는 40.5nJ 로 크게 감소하였다. 다만 추가 register 활용과 unrolling 으로 인해 area 와 power 는 shift-add 단독 구조보다 증가하였다.

Instruction 재배치 및 forwarding unit 제거를 적용한 구조는 clock period 가 2.55ns 로 감소하였다. forwarding mux 가 ALU critical path 에서 제거되어 timing 이 개선되었으며, Average cycle 은 3793 으로 4x4 tile 구조보다 약간 증가했지만, clock period 감소 효과로 execution time 은 9.67 μ s 로 개선되었다. Power 는 4.16mW, area 는 34272.76, energy 는 40.3nJ 이다.

최종 구조는 branch predictor 를 추가 최적화하여 area 와 energy 를 더욱 줄였다. B 행렬 비트 체크 분기(beq x7/x1/x6)는 데이터 의존적 랜덤 패턴이므로 예측 대상에서 제외하고, 루프 제어 분기(beq x2,x3 / beq x5,x1 등)만 예측 대상으로 한정하였다. 또한 루프 종료 분기의 패턴이 7NT→1T 로 단순하여

LHIST 를 2bit 에서 1bit 로 축소하였다. 이를 통해 PHT 를 64 entry 에서 32 entry 로, LHIST 를 16×2bit 에서 16×1bit 로 줄여 predictor storage 를 160bit 에서 80bit 로 50% 감소시켰다. 최종 PPA 는 clock period 2.55ns, power 3.92mW, area 32291.44, average cycle 3793, execution time 9.67μs, energy 37.9nJ 로 가장 낮은 energy 를 달성하였다.

6. 설계 Trade-off 분석

본 설계에서 적용한 최적화는 성능, 면적, 전력 사이의 trade-off 를 가진다. Pipeline 구조는 throughput 을 높이고 clock period 를 줄일 수 있지만, pipeline register 가 추가되므로 sequential area 가 증가한다. 또한 branch 또는 jump 가 발생하면 flush 가 필요하고, data dependency 가 발생하면 forwarding 또는 stall 처리가 필요하다.

4×4 tile 및 unrolling 은 cycle 수를 크게 줄일 수 있으나, instruction memory 사용량과 register 사용량이 증가한다. 특히 16 개의 누산기를 사용하는 구조는 memory 접근을 줄이는 대신 register pressure 를 증가시키며, 코드 길이 증가로 IMEM 크기 요구량이 증가할 수 있다.

Branch predictor 는 반복문이 많은 GEMM 에서 control hazard penalty 를 줄이는 장점이 있다. 그러나 predictor table 과 update logic 이 추가되므로 area 와 power 가 증가할 수 있다. 또한 예측이 틀릴 경우 flush 가 발생하여 penalty 가 생긴다. 따라서 branch predictor 는 예측 대상 branch 를 제한하여 적용하는 것이 중요하다.

Forwarding unit 은 RAW hazard 에 의한 stall 을 줄일 수 있지만, ALU 입력 앞에 mux 와 비교 logic 을 추가하므로 critical path 가 길어지는 단점이 있다. 본 설계에서는 forwarding unit 을 완전히 제거하고, and 명령어 3 개를 beq 보다 먼저 배치하는 instruction scheduling 으로 이를 대체하였다. 이를 통해 gap 을 3 이상으로 확보하여 forwarding 없이 register file 에서 직접 읽을 수 있었고, critical path 단축으로 clock period 를 2.55ns 로 줄일 수 있었다.

최적화	장점	단점
5-stage pipeline	instruction throughput 향상, clock period 감소 가능	pipeline register 및 control 복잡도 증가
4×4 tile	cycle 수 대폭 감소, memory 접근 감소	code size 및 register pressure 증가
Branch prediction	loop branch penalty 감소	predictor table 및 flush 복구 logic 증가
A/B masking	lbu 없이 8-bit input 처리 가능	MEM path 에 masking logic 추가
ALU case 제거	area/control 단순화, timing 개선 가능	full RV32I 범용성 감소
Instruction scheduling	stall 감소, pipeline 효율 개선	assembly 와 microarchitecture 의 결합도 증가

7. 제약 조건 준수 여부

본 설계는 프로젝트에서 요구하는 제약 조건을 만족하도록 구성하였다. 첫째, 별도의 외부 GEMM accelerator 를 추가하지 않았다. GEMM 연산은 모두 CPU 가 instruction memory 에서 명령어를 fetch 하고, datapath 를 통해 실행하는 방식으로 수행된다. 즉, CPU 외부에 matrix multiplier unit 을 붙인 구조가 아니다.

둘째, top-level interface 와 memory map 을 유지하였다. A, B, C, STATUS 의 기본 주소는 변경하지 않았고, 연산 완료 후 STATUS 주소에 1 을 저장하도록 하였다. 이는 testbench 가 STATUS 를 감지한 뒤 C 영역 결과를 golden model 과 비교하는 검증 방식과 호환된다.

셋째, custom instruction 을 추가하지 않았다. 곱셈은 MUL instruction 이나 custom ISA 가 아니라, ADD, AND, BEQ, JAL 등의 기본 명령어 조합으로 구현하였다. bit 기반 shift-add 알고리즘을 사용하여 RV32I subset instruction execution flow 내에서 GEMM 연산을 수행하였다.

제약 조건	준수 여부	설명
Top-level port 변경 금지	준수	RISCVMULTI 는 기존 PC, INSTR, MEMWRITE, ALURESULT, WRITEDATA, READDATA 인터페이스 유지
외부 GEMM accelerator 금지	준수	모든 GEMM 연산은 instruction execution flow 내부에서 수행
Custom instruction 금지	준수	MUL/custom instruction 없이 ADD/AND/BEQ/JAL 기반 shift-add 사용
Memory map 유지	준수	A=0x000, B=0x100, C=0x200, STATUS=0x300 유지
STATUS 완료 신호	준수	연산 완료 후 STATUS 주소에 1 저장
Testbench 검증	준수	STATUS 감지 후 C matrix 를 golden model 과 비교

8. 결론

본 프로젝트에서는 8×8 GEMM 연산을 효율적으로 수행하기 위해 RISC-V 기반 프로세서의 assembly code 와 microarchitecture 를 함께 최적화하였다. 5-stage pipeline 구조를 통해 instruction throughput 을 향상시켰고, branch prediction 과 flush control 을 적용하여 control hazard 를 완화하였다. 또한 forwarding unit 제거, A/B 8-bit masking, ALU case 제거 및 alucontrol 2-bit 축소, branch predictor 최적화 등을 통해 datapath 와 control path 를 GEMM workload 에 맞게 개선하였다.

Assembly 측면에서는 단순 반복덧셈 대신 bit 기반 shift-add 방식을 적용하고, 4×4 tile 단위 계산으로 16 개의 누산기 register 를 활용하였다. 또한 and 명령어 선집결을 통한 instruction scheduling 으로 forwarding unit 을 완전히 제거하였으며, branch predictor 에서 랜덤 패턴의 B-matrix bit-check 분기를 예측 대상에서 제외하고 LHIST 를 1bit 로 축소하여 predictor storage 를 50% 감소시켰다. 최종 구조는 clock period 2.55ns, average cycle 3793, execution time 9.67μs, energy 37.9nJ 수준의 결과를 보였다.

결론적으로 본 설계는 단순한 matrix multiplication accelerator 가 아니라, RISC-V instruction execution flow 를 유지하면서 GEMM 실행 패턴에 맞게 datapath, control path, assembly scheduling 을 함께 최적화한 processor 구조이다. 다만 full RV32I 전체 명령어 지원보다는 GEMM 에 필요한 subset 중심으로 최적화했기 때문에 범용성과 성능/면적 사이의 trade-off 가 존재한다.